TD1 – Electronique numérique

Représentation des nombres

Notes

- Le but du TD est de manipuler différentes représentations des *nombres* et notamment maîtriser les changements de base *sans calculatrice*.
- En dehors du TD, on vous suggère de vérifier que vous êtes suffisamment adroits pour les réaliser également *avec* calculatrice ou à l'aide de votre langage de programmation préféré.
- *Tous les exercices ne seront pas corrigés en séances.*

1 Compter en base 2 et base 16

1. Compter de 0 à 15 en binaire, puis en hexadécimal. *(On conservera ces valeurs dans un tableau pour la durée du TD)*
2. Ecrire les puissance de 2^n pour n variant de 0 à 10. En déduire une valeur approchée de $\log_{10} 2$, puis $\log_{10} 5$. Avec un procédédé similaire, trouver une valeur approchée de $\log_{10} 3$
3. Calculer la valeur décimale de 10101011_2 en passant par les puissances de 2 et en passant par les puissances de 16
4. Ecrire les puissance de 2^{-n} pour n variant de 1 à 4

2 Conversions entre bases

1. Convertir le nombre 39 en binaire.
2. Convertir le nombre $(0.375)_{10}$ en binaire.
3. En déduire la représentation binaire du le nombre $(39.125)_{10}$
4. Convertir le nombre $(0.2)_{10}$ en binaire. Que remarquez-vous ?
5. Convertir $(345.158)_{10}$ en octal
6. Convertir $(389)_{10}$ en hexa
7. Convertir 011001111011101_2 en hexa

8. Convertir $(10110001101011.1111)_2$ en octal
9. Convertir $(10110001101011.111101)_2$ en hexa
10. Le nombre 101110001_2 est écrit en BCD (décimal codé binaire). Convertir le nombre décimal obtenu en binaire pur. Combien de bits sont alors nécessaires ? Conclure.

3 Nombres signés en complément à 2

Rappels

Jusqu'ici, nous nous sommes intéréssés aux nombres positifs. Il existe plusieurs manières de représenter un nombres négatif n. La manière la plus intuitive serait de dédier un bit (le plus à gauche) pour préciser le signe du nombre : par exemple on pourrait choisir 0 pour le signe + et 1 pour le signe −. Malheureusement, cette représentation possède deux défauts majeurs : il existe alors deux représentations du zéro et surtout, il faut modifier l'algorithme d'addition. Les architectes des ordinateurs ont retenu une autre représentation plus pratique : le **complément à 2**. C'est cette représentation qui régit le fonctionnement des *unités arithmétiques* des ordinateurs sur lesquelles nous reviendrons plus tard.

Avec k bits, on peut représenter l'ensemble des nombres

$$[-2^{k-1}, \ldots, 2^{k-1} - 1]$$

Le complément à 2 d'un entier n est obtenu par le calcul de la quantité $2^k - n$: on comprend ici que le terme "complément à 2" est un raccourci pour "complément à 2^k ". On peut également vérifier aisément que :

— Le complément à 2 du complément à 2 d'un entier n est n lui-même.
— Le complément à 2 de zéro est zéro (on néglige la retenue qui dépasse la taille fixée).

On doit retenir la formule de passage en complément à 2 d'un nombre négatif $-X$:

$$-X = \overline{X} + 1 \tag{1}$$

La barre figurant au dessus du X signifie "inversion de tous les bits" (complément à 1).

Si l'entier n est codé sur k bits $(s_{k-1} \ldots s_0)$ en complément à 2, on a alors :

$$n_{10} = -s_{k-1}2^{k-1} + \sum_{i=0}^{k-2} s_i 2^i \tag{2}$$

Exercices

1. Déterminer la plage des valeurs des entiers signés sur $k = 4$ bits. Représenter ces valeurs en binaire. Quelles sont les représentations des valeurs extrêmes ? de 0,1, et -1 ?

2. Quel est le complément à 2 du nombre 8, codé sur 6 bits ? Utiliser la formuler 1 et vérifier le résultat en appliquant la formule 2.
3. Soustraire : $91_{10} - 108_{10}$ en passant par le complément à 2
4. Ecrire les nombres suivants en binaire sur 6 bits : $+24, +31, -24, -31$. Passez ensuite à 7 bits. Que remarquez-vous ?
5. Soustraire 1100 0101 0100 0000 - 101000 1111 1111 - 110 1111 1110 - 10 1110

4 IEEE 754

Rappel : norme IEEE 754 (simple précision, 32 bits) Un flottant est codé par trois champs : 1 bit de signe s, 8 bits d'exposant e et 23 bits de mantisse m. La valeur représentée est, pour la plupart des nombres,

$$V = (-1)^s \times 2^{e-127} \times (1 + m),$$

où $m = \sum_{i=1}^{23} b_i 2^{-i}$ est la partie fractionnaire. On dit que l'exposant est représenté en excédent (ou biais) à 127. La mantisse est toujours normalisée avec un 1 implicite (gain de place), sauf pour les cas particuliers suivants :

— si $e = 255$ et $m = 0$, alors $V = \pm\infty$ (selon s) ;
— si $e = 255$ et $m \neq 0$, alors $V =$ `NaN` (not a number) ;
— si $e = 0$ et $m = 0$, alors $V = \pm 0$;
— si $e = 0$ et $m \neq 0$, il s'agit d'un nombre dénormalisé, dont la valeur est $V = (-1)^s \times 2^{-126} \times m$.

Pour convertir un décimal en IEEE 754, on écrit sa valeur absolue en binaire, on la normalise sous la forme $1{,}XXXXX \times 2^n$, puis on pose $e = n + 127$ et l'on conserve les 23 premiers bits de la partie fractionnaire comme mantisse (avec arrondi éventuel). La conversion inverse s'obtient par la formule ci-dessus, en lisant les champs binaires.

Représenter le nombre 238,4375 dans sa représentation IEEE 754 (32 bits) en virgule flottante.